

DP Computer Science: B 3.1.2 Designing Classes

1. Core Concepts & Learning Objectives

Learning Intentions

By the end of this lesson, students will be able to:

- Define what a **class** is and identify its key components (**attributes** and **methods**) based on given application requirements
 - Explain the purpose of **UML (Unified Modeling Language) class diagrams** in software design
 - Recognise and interpret the basic symbols and structure of a UML class diagram, including **class name, attributes, methods, and visibility**
 - Create **basic UML class diagrams** to represent the design of simple classes with their attributes and methods
 - Understand how UML class diagrams aid in effective software design and planning
-

Key Vocabulary

Term	Definition
Class	A blueprint or template for creating objects; defines the attributes and methods that objects of that class will have
Object	A specific instance of a class (e.g., a specific car made from the Car blueprint)
Attribute	A characteristic or property of a class (e.g., colour, brand, speed)
Method	An action or behaviour that a class can perform (e.g., accelerate, brake)
UML (Unified Modeling Language)	A standardised visual language used to design, document, and communicate software

Term	Definition
	architecture
UML class diagram	A diagram that visually represents the structure of a class, showing its name, attributes, methods, and relationships
Visibility	Access control for attributes and methods (public + or private -)
Abstraction	Focusing on essential characteristics while ignoring irrelevant details
Encapsulation	Bundling data and methods that operate on that data; hiding internal state

Relevant ATL Skills

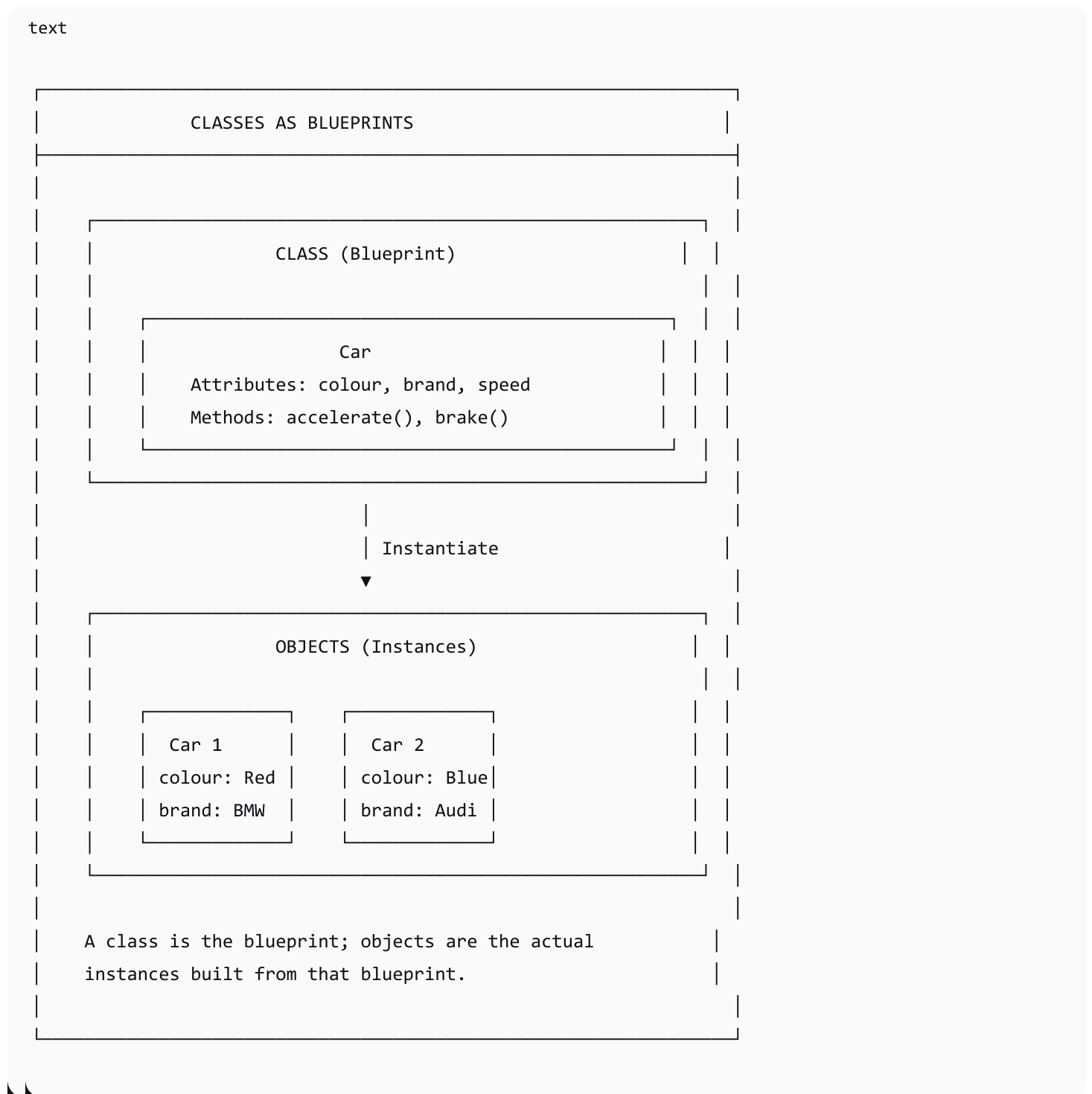
Skill	Description
Thinking (Abstraction)	Focusing on essential characteristics of real-world entities to represent them as classes
Thinking (Decomposition)	Breaking down application requirements into manageable classes and identifying their parts
Communication (Visual Communication)	Using UML class diagrams as a visual tool to communicate class design
Communication (Technical Vocabulary)	Using appropriate terminology to discuss class designs

2. Introduction: Classes as Blueprints

What is a Class?

A class is a blueprint or template for creating objects. It defines the attributes (data) and methods (behaviours) that objects of that class will have.

The Blueprint Analogy



Class vs. Object

Concept	Definition	Example
Class	A blueprint or template	Car
Object	A specific instance of a class	myCar (a red BMW)
Attribute	A property of the class	colour , brand , speed
Method	An action of the class	accelerate() , brake()

Car Example

Aspect	Description
Class Name	Car
Attributes	colour , brand , speed , fuelLevel
Methods	accelerate() , brake() , refuel()

python

In Python code:

```
class Car:
    def __init__(self, colour, brand):
        self.colour = colour
        self.brand = brand
        self.speed = 0

    def accelerate(self):
        self.speed += 10

    def brake(self):
        self.speed -= 5
```

3. Introduction to UML Class Diagrams

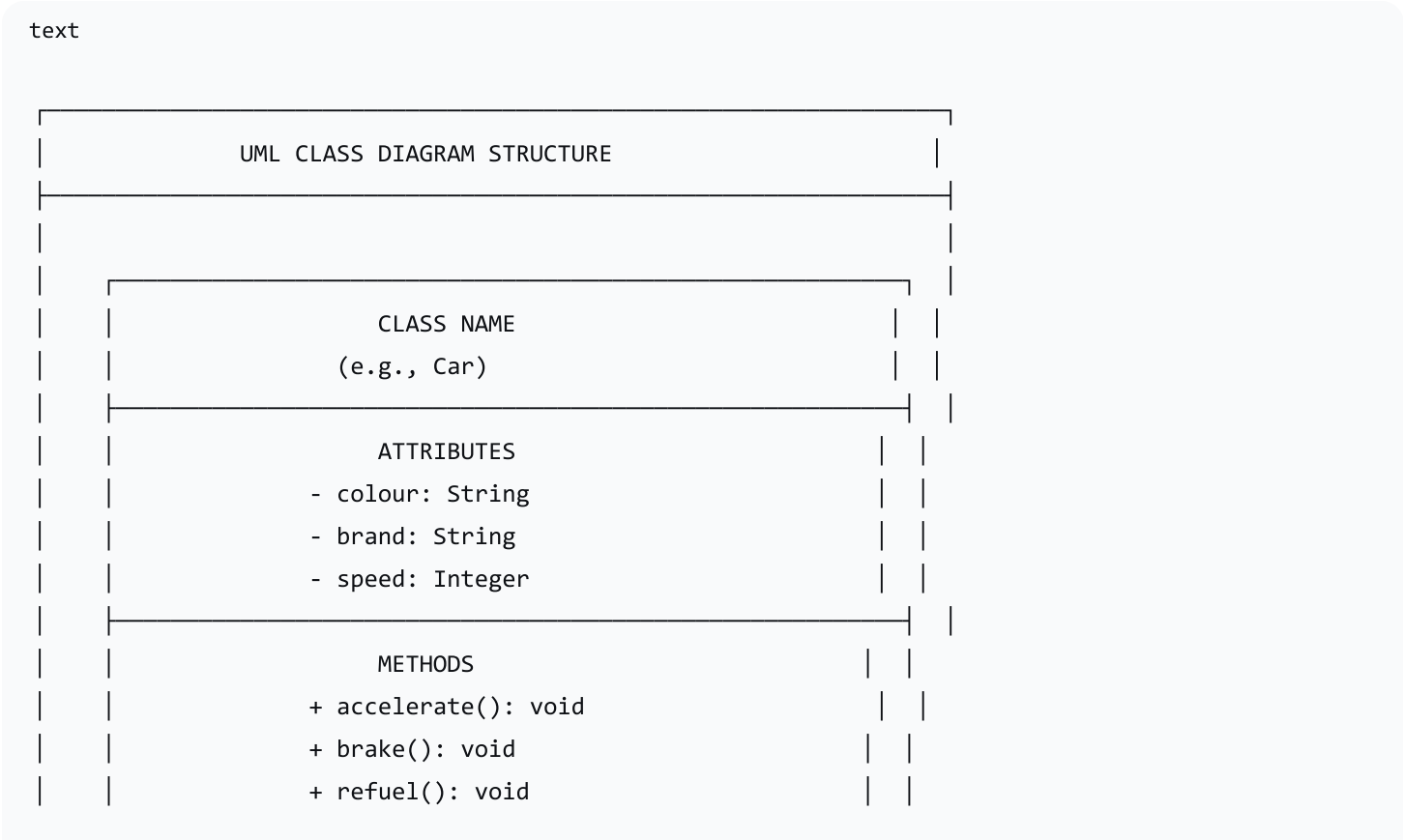
What is UML?

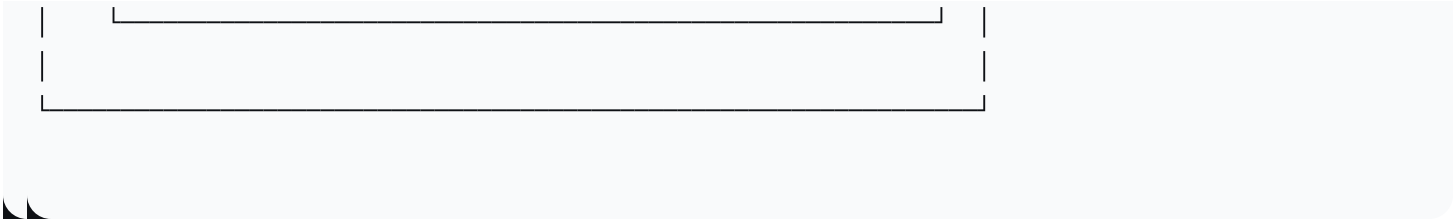
UML (Unified Modeling Language) is a standardised visual language used to design, document, and communicate software architecture.

Why Use UML?

Reason	Description
Visual Communication	Provides a common language for developers, designers, and stakeholders
Design Clarity	Helps clarify the structure of the software before coding
Documentation	Serves as documentation for the system
Planning	Facilitates planning and identifying missing components
Collaboration	Enables teams to work together effectively

Basic UML Class Diagram Structure





Visibility Symbols

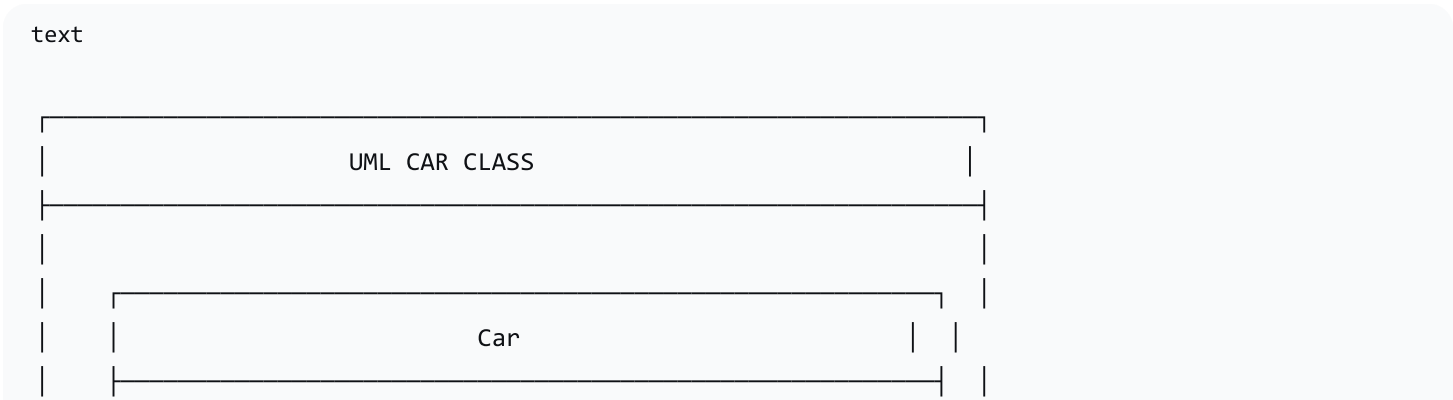
Symbol	Meaning	Description
+	Public	Accessible from outside the class
-	Private	Accessible only within the class
#	Protected	Accessible within the class and subclasses

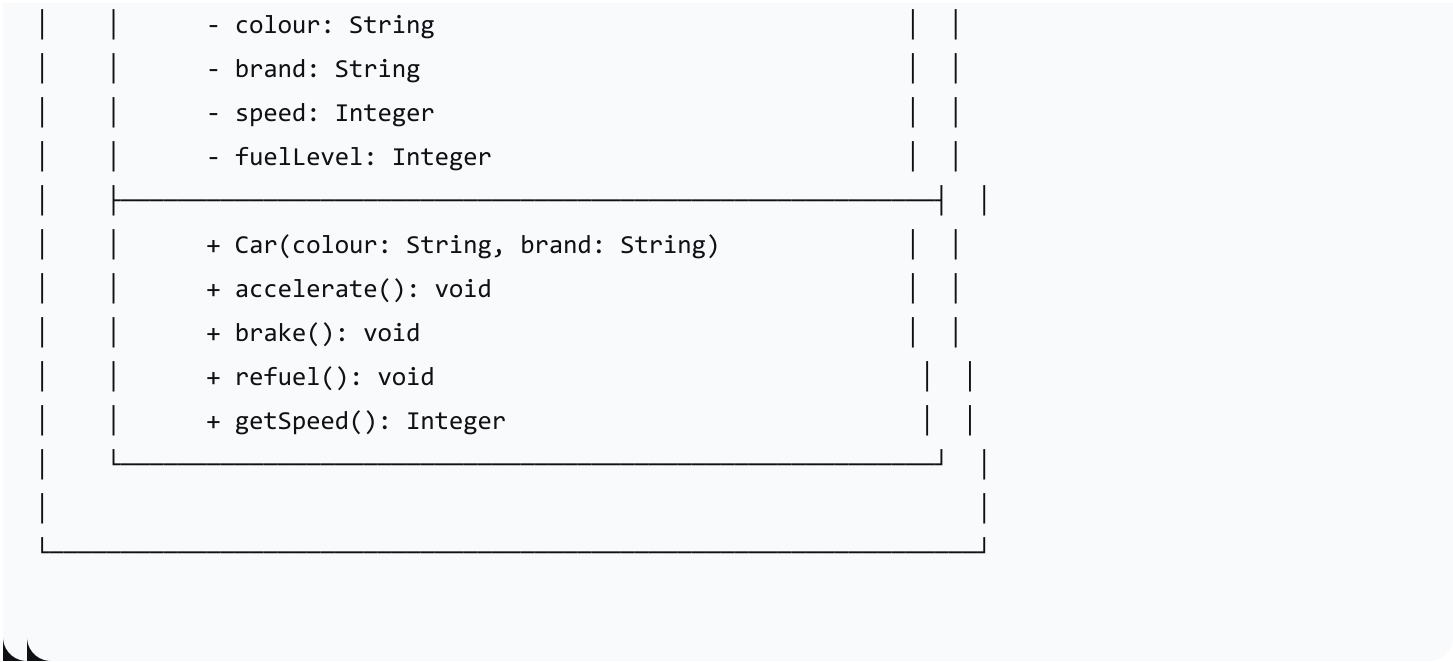
UML Notation Format

Element	Format	Example
Attribute	visibility name: type	- colour: String
Method	visibility name(parameters): returnType	+ accelerate(): void

4. Car Class in UML

UML Diagram for Car





Explanation of UML Components

Component	Explanation
Class Name	Car (first compartment)
Attributes	- colour: String , - brand: String , etc. (second compartment)
Methods	+ accelerate(): void , etc. (third compartment)
Visibility	- = private, + = public
Constructor	+ Car(colour: String, brand: String)

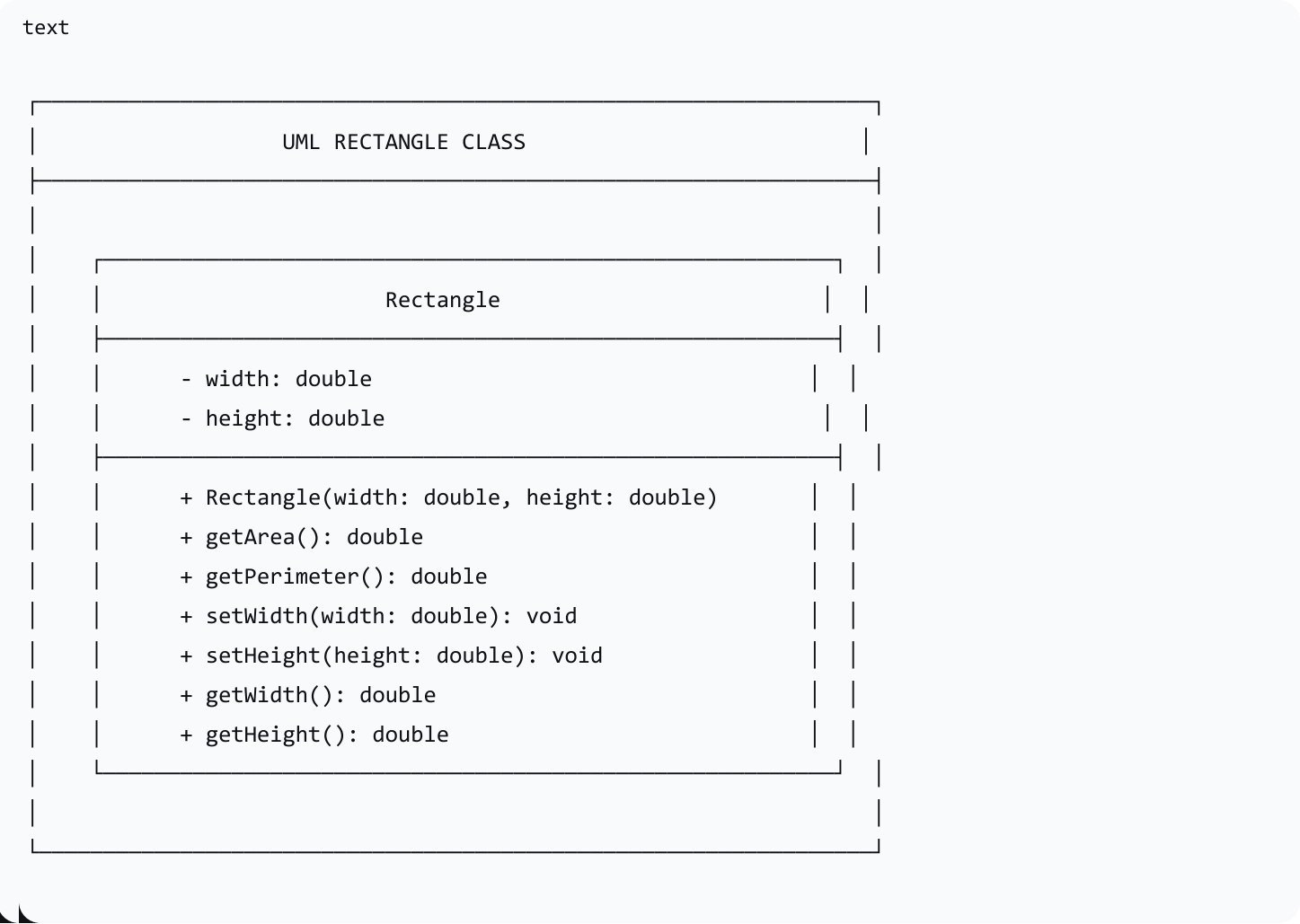
5. Think-Pair-Share Activity: Rectangle Class

Instructions

1. **Think:** Individually brainstorm:
 - What attributes does a rectangle need?
 - What methods should a rectangle have?
2. **Pair:** Discuss your ideas with a partner.

3. **Share:** Share your ideas with the class.

Possible Solutions



Attributes and Methods Explained

Attribute	Type	Description
width	double	The width of the rectangle
height	double	The height of the rectangle
Method	Return Type	Description
getArea()	double	Calculates the area (width × height)

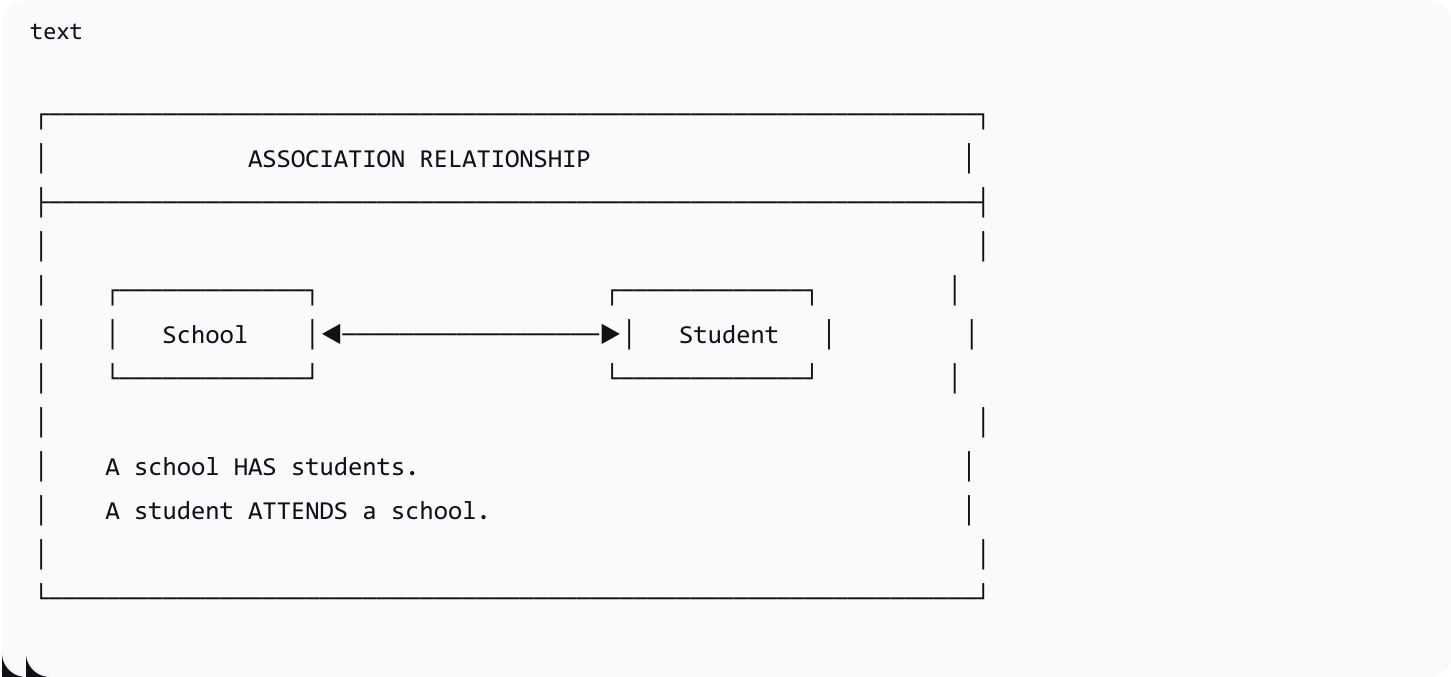
Method	Return Type	Description
<code>getPerimeter()</code>	double	Calculates the perimeter ($2 \times (\text{width} + \text{height})$)
<code>setWidth()</code>	void	Updates the width
<code>setHeight()</code>	void	Updates the height
<code>getWidth()</code>	double	Returns the width
<code>getHeight()</code>	double	Returns the height

6. Introduction to Relationships

What are Relationships?

Relationships show how classes are connected to each other. The most common relationship is association.

Association



Types of Relationships

Relationship Type	Description	UML Notation
Association	A general connection between classes	
Inheritance	A "is-a" relationship (subclass)	
Aggregation	A "has-a" relationship (whole-part)	
Composition	A strong "has-a" relationship (part cannot exist without whole)	

Note: For this introductory lesson, focus on association. More advanced relationships will be covered later.

7. Lesson Sequence

Lesson Plan (60 minutes)

Phase	Time	Activity
Introduction	5 min	Welcome; learning objectives
Classes, Attributes, Methods	10 min	Explanation with Car example
UML Introduction	10 min	What is UML; basic diagram structure
Visibility & Example	10 min	Public/private; UML for Car class
Relationships	5 min	Brief introduction to association
Think-Pair-Share	5 min	Brainstorm Rectangle class attributes and methods
Worksheet	10 min	Students work individually on worksheet

Phase	Time	Activity
Summary	5 min	Review; Q&A; preview next lesson

Worksheet Exercises

Exercise	Description
Part 1	Identify the class, attributes, and methods from a scenario
Part 2	Interpret a UML diagram and describe the class
Part 3	Draw a UML diagram for a given class (e.g., Student, Book)

8. Assessment Activities

Assessment Type	Description
Class Participation	Observe engagement and contributions
Think-Pair-Share	Assess understanding of attributes and methods
Worksheet	Evaluate ability to interpret and create UML diagrams
Exit Ticket	Gauge understanding and identify areas needing clarification

Sample Exit Ticket Questions:

1. What is a class?
2. What is the difference between an attribute and a method?
3. What does a UML class diagram show?
4. What is the difference between **+** and **-** in UML?

9. Differentiation

Level	Strategy
Support	Provide visual aids, real-world examples, and scaffolded worksheets
Challenge	Explore inheritance and relationships; design more complex classes

10. Resources

- **Presentation** (Slide Deck)
- **eBook** (B3.1.2 with examples)
- **Worksheet**: UML class diagram exercises
- **Worksheet Answers**
- **Quizlet** (B3.1.2 vocabulary flashcards)

11. Summary: Key Takeaways

Concept	Key Point
Class	A blueprint for creating objects
Object	A specific instance of a class
Attribute	A property of a class
Method	An action of a class
UML	A visual language for designing software
Visibility	 = public,  = private
UML Structure	Class name, attributes, methods

text

KEY REMINDERS

- ✓ Class = blueprint for objects
- ✓ Object = specific instance
- ✓ Attributes = data (properties)
- ✓ Methods = actions (behaviours)
- ✓ UML = visual design language
- ✓ + = public (accessible outside)
- ✓ - = private (hidden inside)

"A class is a blueprint—it defines the structure and behaviour of objects before they are created."